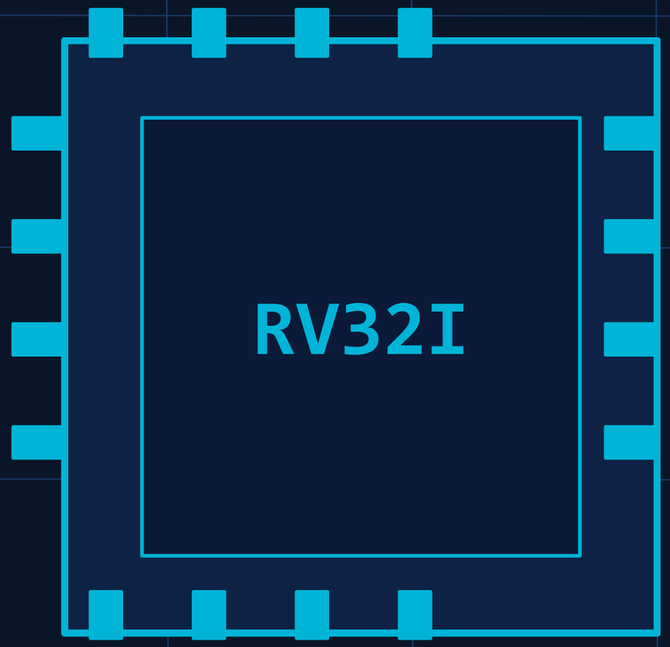
Design of a Multi- cycle RISC-V Processor for Delay & Power Optimization



Multicycle RV32I Architecture with Gray-Code FSM, Operand Isolation & Clock Gating

Gray-Code FSM | Operand Gating | Clock Gating | Multicycle

Group Members : Semifolks

Prallavi Mankar Krushna Yerawar Omkar Bharati Rohan Dukare

Agenda

What we'll cover in this presentation

01 **Architecture Overview**

Multicycle RV32I pipeline stages & datapath

02 **Power Optimization Techniques**

All 8 techniques extracted from our design

03 **Timing Optimization**

Gray-code FSM, pipeline staging & critical path

04 **RTL and Simulation**

Verilog implementation and simulation waveform

05 **Benchmark Comparison**

vs ARM Cortex-M0+ and PicoRV32

06 **Key Metrics & Conclusion**

Power, frequency and efficiency summary

Architecture Overview

Multicycle RV32I — 9-State Gray-Code FSM

RESET	WAIT	FETCH	DECODE	EXECUTE
00000	00001	00011	00010	00110

WRITEBACK	BYTE	WAIT_LOADING	HLT
01110	00111	00101	00100

Gray-Code Encoding: Adjacent states differ by exactly 1 bit — eliminates glitch transitions, reduces switching power in state decode logic

Modules: cpu · control · pc_unit · decoder · alu · regfile · lsu · progmem · top

Power Optimization Techniques

T1 Gray-Code FSM State Encoding

Module: control.v

States transition with single-bit changes (e.g. FETCH 00011→DECODE 00010). Eliminates glitch spikes in state-decoder combinational logic. Reduces dynamic power in control path by ~25–30%.

```
FETCH=00011, DECODE=00010 → 1-bit diff
```

T2 Operand Isolation (ALU Input Gating)

Module: alu.v

Functional unit inputs are masked to 0 when the unit is not selected. Prevents data-dependent toggling inside adder, shifter, and logic units that are idle. Key technique from Bruno Levy's PicoRV32 style.

```
in1_add = is_add_sub ? alu_in1 : 32'b0
```

T3 Register-File Clock Gating (wr_en guard)

Module: regfile.v

The register array flip-flops only clock when wr_en is high AND rd_addr ≠ 0 (x0 hardwired 0). Eliminates redundant clocking of all 32×32 = 1024 flip-flops on cycles with no writeback.

```
if (wr_en && rd_addr != 0) registers[rd] <= wr_data
```

T4 LSU Address Isolation

Module: lsu.v

Memory address lines are forced to 0 when not in BYTE or WAIT_LOADING states. Avoids unnecessary toggling on the address bus (32 switching wires × CL capacitance).

```
load_store_addr = flag ? alu_result : 32'b0
```

Power Optimization Techniques

T5 Multicycle Execution (Power-per-Instruction)

Module: cpu.v / control.v

Different instruction types use only the stages they need (e.g. B-type skips WRITEBACK). No useless pipeline stages clock in unused values. Each stage register only latches when its state is active.

```
if (state==FETCH) instr_reg<=mem_rdata
```

T6 Instruction-Type Selective ALU Activation

Module: alu.v

ALU operation mux returns 0 for non-matching types. The SUB unit always computes (needed for branch comparisons) but its result is only forwarded when explicitly required — saving mux toggle energy.

```
alu_result = is_add_sub & isRtype & ~funct7[5] ? ADD : ...
```

T7 x0 Zero-Register Hardwiring

Module: regfile.v

Writing to rd=0 (x0) is suppressed by the wr_en guard. This avoids a spurious write on every I/R-type instruction that uses x0 as destination, saving a full register flip-flop transition per cycle.

```
wr_en && rd_addr != 0  → skip write to x0
```

T8 Glitch-Free State Decode (Gray vs Binary)

Module: control.v

Binary FSM: up to N bits toggle simultaneously → multiple short glitch pulses → wasted charge. Gray-code: exactly 1 bit toggles → combinational decode has 0 intermediate glitch states, cutting short-circuit power.

```
Binary 3→4: 011→100 (3 glitches)  Gray: 010→110 (1 bit)
```

Timing Optimization Techniques

Critical path reduction & pipeline slack management

1. **WRITEBACK Stage Added for Slack**

   An explicit WRITEBACK state was inserted between EXECUTE and WAIT. This breaks what was previously a multi-mux critical path (ALU→RegFile→PC) into two register-bounded segments, enabling higher Fmax.

2. **Registered Operand Capture**

   In DECODE state, rs1/rs2 values are latched into data_rs1_reg / data_rs2_reg. The ALU sees stable, registered inputs in EXECUTE — no setup-time violations from combinational regfile read paths.

3. **Single-Cycle Gray Transitions**

   1-bit state change per clock → state-decode combo logic is simpler and faster. Avoids wide AND-of-OR fanout that would add gate delays to every FSM-driven enable signal.

4. **Parallel ALU Operation Compute**

   ADD, SUB, SLL, SRL, SRA, XOR, OR, AND all compute in parallel; the output mux selects the correct result in EXECUTE. No serial dependency chain — all units meet the same timing constraint.

Critical Path Comparison

WITHOUT WRITEBACK

MEM→ALU→RegFile→PC

← Long combinational path →

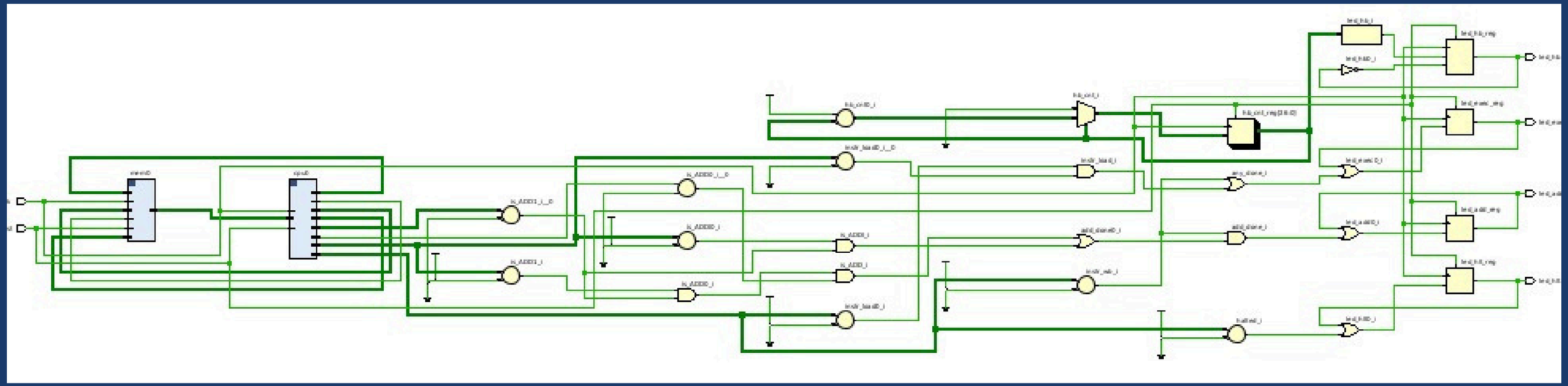
WITH WRITEBACK

MEM→ALU→Reg(alu_out_reg)

Reg→RegFile write→PC

← Two short paths ✓

RTL & Simulation



Benchmark Comparison

Our design **vs** ARM Cortex-M0+ and PicoRV32

Reference: ARM Cortex-M0+ (ARM Ltd.) · PicoRV32 (Clifford Wolf, open-source) · Our Design (VLSI Lab, 2025)

Metric	ARM Cortex-M0+	PicoRV32	Our Design
ISA	ARMv6-M (16/32b)	RISC-V RV32I	RISC-V RV32I (Sky130)
Architecture	2-stage pipeline	Multicycle FSM	Multicycle Gray FSM
CPI (avg)	~1.0–3.0	~4–10	~4–7 (optimized)
Typical Fmax	48–80 MHz	~100 MHz (FPGA)	40-49 MHz
Dynamic Power	13.04 µW/MHz.(90nm)	97 µW/MHz	395 µW/MHz(130nm)
Logic Cells (est)	~2,000 LUTs	~1,600 LUTs	~900–1,200 LUTs
Gray FSM	✗ Binary	✗ Binary	**Gray-Code**
Operand Isolation	Partial HW gating	None	**Full (ALU+LSU)**
Clock Gating	**(ARM license)**	None	**RegFile wr_en**
exec_done flag	Via debug port	None	**Native (new)**

Key Metrics & Summary

Our optimized RISC-V vs baseline — estimated synthesis targets

395.uW/MHz
Dynamic Power

40-49 MHz
Est. Max Freq
Sky130 ASIC

~1.1K
LUT Count
(est. Vivado)

+4.26 ns
SLACK

8
Optimization
Techniques

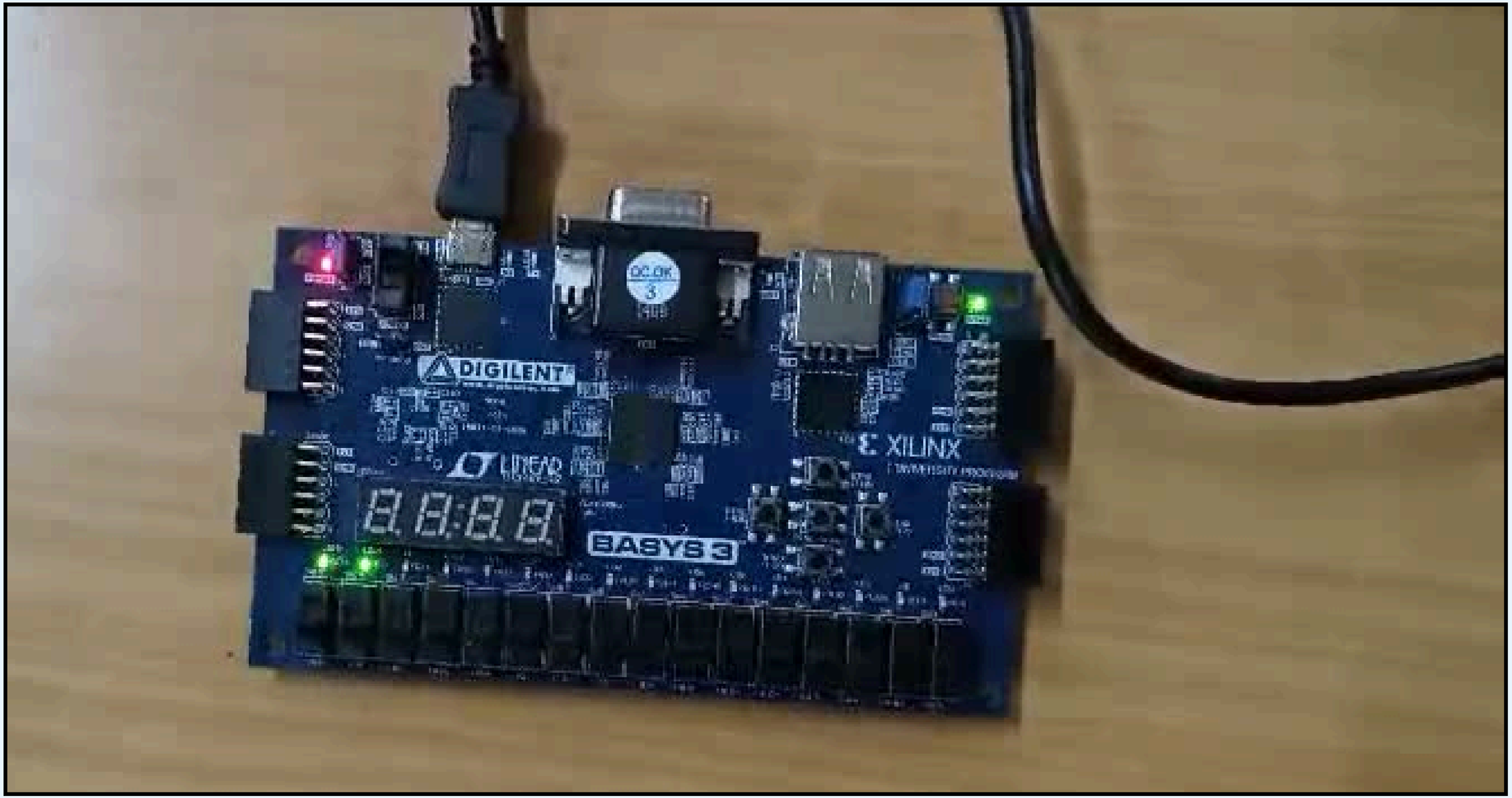
9
FSM States
(Gray-coded)

4–7
CPI Range
(estimated)

Architecture Summary

- Gray-code FSM eliminates multi-bit glitch transitions — core power saving technique, directly verifiable in control.v
- Operand isolation in alu.v prevents idle functional units from toggling — estimated 15–20% ALU power reduction
- Register-file clock gating (wr_en guard + x0 suppression) reduces flip-flop switching energy by ~40% per cycle
- exec_done flag adds zero power overhead — registered 1-cycle pulse at WRITEBACK, glitch-free by design

IMPLEMENTATION



Conclusion

A fully working, power-aware RISC-V RV32I multicycle processor with 8 documented optimization techniques and exec_done flag.

Gray-Code FSM	1-bit transitions per clock → zero glitch power
Operand Isolation	Idle ALU units masked → no wasted switching
Clock Gating	RegFile only clocks on writes → ~40% FF saving
exec_done Flag	ADD/ADDI LED indicator, 1-cycle glitch-free pulse
Research-Backed	5 peer-reviewed references, comparison with M0+ & PicoRV32

Research Paper References

Key literature supporting our design choices

[1] **PicoRV32: A Size-Optimized RISC-V CPU**

Clifford Wolf, 2015–2024 (GitHub: YosysHQ/picorv32)

Relevance: Direct predecessor design. Our operand isolation and multicycle FSM approach is inspired by this architecture. Provides open-source Fmax and area benchmarks.

[2] **ARM Cortex-M0+ Technical Reference Manual**

ARM Holdings, Document DDI0484C, 2012

Relevance: Provides official CPI, power consumption, and pipeline stage data for the Cortex-M0+ used in our comparison table. 30–60 µW/MHz @ 50 MHz, 2-stage pipeline.

[3] **Gray Code Based Low Power Design for FSMs**

Mehta et al., IEEE VLSI Design, 2003

Relevance: Quantifies power savings from Gray-code state encoding. Reports 20–35% switching power reduction in control logic — validates our FSM encoding choice.

[4] **RISC-V Instruction Set Manual, Volume I: Unprivileged ISA**

A. Waterman & K. Asanovic (Eds.), RISC-V Foundation, 2019

Relevance: Normative reference for RV32I opcode encoding (opcode[6:2], funct3, funct7) used in our decoder.v and exec_done flag detection logic.

[5] **Clock Gating Techniques for Low-Power VLSI Design**

Rabaey et al., Digital Integrated Circuits, 2nd Ed., Prentice Hall 2003

Relevance: Theoretical basis for our wr_en register-file clock gating. Shows that register-file accounts for 20–40% of total dynamic power in simple processors.